

EPIC 1: Secure User Login, Personalized Dashboard & Diagnosis History Management

PILE 1 — User Registration

US1: Account Creation

As a new user, I want to create an account using email or phone so that I can securely access the system.

Acceptance Criteria

- AC1.1: Email/phone format validated
- AC1.3: User stored in DB

Test Cases

- TC1: Register with valid data → success
- TC2: Weak password → error
- TC3: Duplicate email → error

Design Notes

- Registration form → POST /auth/register
- DB fields: name, email, phone, password_hash

Plan (Sprint Tasks)

- Create model
- Implement validator
- Encrypt password

PILE 2 — Secure Login

US2: Login Authentication

In order to access my personalized dashboard as a registered user, I want to log in using valid credentials.

AC

- AC2.1: Correct credentials → login
- AC2.2: Wrong credentials → error
- AC2.4: Session stored securely

Tests

- TC1: Valid login
- TC2: Invalid login
- TC3: Missing fields

Design

- POST /auth/login
- JWT + refresh token

Plan

- Setup token service
- Integrate with frontend

PILE 3 — Logout

US3: Session Termination

As a user, I want to log out of the system.

AC

- AC3.1: Token invalidated
- AC3.2: Redirect to login page

Tests

- TC1: After logout → cannot access protected routes

Design

- Clear localStorage/session

Plan

- Implement logout endpoint
- Block dashboard access

PILE 4 — Forgot Password

US4: Reset Password

In order to regain access to my account as a user, I want to reset my password securely.

AC

- AC4.2: Secure reset link
- AC4.3: New password validation

Tests

- TC1: OTP verification works
- TC2: Reset link expiry

Design

- Two-step reset flow

Plan

- Build OTP generator

- OTP/email verification

PILE 5 — Profile Management

US5: Update Profile

As a user, I want to update my profile details so that the system can personalize my experience.

AC

- AC5.1: Editable name & language

Tests

- TC1: Update → DB updated
- TC2: Invalid inputs → error

Design

- PATCH /user/profile

Plan

- Create frontend profile page
- Dashboard updates language

PILE 6 — Dashboard Overview

US6: Dashboard Summary

In order to quickly understand my crop health status as a user, I want a dashboard showing recent diagnoses.

AC

- AC6.2: Load < 2 seconds
- AC6.3: Display notifications

Tests

- TC1: Correct data displayed
- TC2: No data → show placeholder

Design

- Cards layout for diagnoses

Plan

- Create dashboard API
- AC6.1: Show last 5 diagnoses

PILE 7 — Notification Panel

US7: Notifications9

As a logged-in user, when I open the dashboard on the home screen, I want to see notifications because I need timely alerts.

AC

- AC7.1: API fetches unread notifications
- AC7.2: Mark-as-read

Tests

- TC1: Notification count correct
- TC2: Mark read works

Design

- Notification list component

Plan

- Build notification API

PILE 8 — Save Diagnosis History

US8: Automatic History Logging

As a user, I want my diagnosis results to be saved automatically so that I can refer to them later.

AC

- AC8.1: Store timestamp, crop, disease, confidence
- AC8.2: Save only after valid diagnosis

Tests

- TC1: Entry saved in DB
- TC2: Missing fields → reject

Design

- history table (user_id FK)

Plan

- Add DB triggers / API

PILE 9 — View History

US9: History Listing

As a user, I want to view my past diagnosis history.

AC

- AC9.1: Sorted newest → oldest
- AC9.2: Pagination after 20 entries

Tests

- TC1: Correct sorting
- TC2: Pagination works

Design

- /history endpoint

Plan

- Build history listing API

PILE 10 — Filter History

US10: Filter/Search

As a user, I want to filter my diagnosis history so that I can quickly find specific records.

AC

- AC10.1: Filters combine together
- AC10.2: Reset filter resets list

Tests

- TC1: Date range filter
- TC2: Crop filter

Design

- Query-based API filtering

Plan

- Implement query params

PILE 11 — Detailed Report View

US11: Report Details

As a user, when I select a history entry from the dashboard, I want to view a detailed report because I need full treatment information.

AC

- AC11.1: Show image, disease, treatment
- AC11.2: Click history → open details page

Tests

- TC1: Correct ID fetch

Design

- /history/:id endpoint

Plan

- Build detail UI

PILE 12 — Download Report

US12: PDF Report

In order to keep records offline as a user, I want to download diagnosis reports as PDF files.

AC

- AC12.1: PDF contains all fields

- AC12.2: Proper formatting

Tests

- TC1: PDF download
- TC2: Missing fields? → fix

Design

- PDF generation service

Plan

- Integrate PDF library

PILE 13 — Delete History Entry

US13: Delete Logs

As a user, I want to delete selected diagnosis records so that I can manage my data privacy.

AC

- AC13.1: Soft delete stored
- AC13.2: Ask confirmation

Tests

- TC1: Entry removed visually

Design

- DELETE /history/:id

Plan

- Add delete API

PILE 14 — History Analytics

US14: Insights & Trends

In order to identify recurring crop issues as a user, I want to see analytics based on my diagnosis history.

.

AC

- AC14.1: Chart of disease frequency
- AC14.2: Crop-wise trends

Tests

- TC1: Chart accuracy

Design

- Pie chart + line chart

Plan

- Build analytics endpoint

PILE 15 — Multi-Device Login Support

US15: Device Sessions

As a user, when I access the system from different devices, I want my sessions to be managed securely because my account must remain protected.

AC

- AC15.1: Session tracking
- AC15.2: Logout from all devices

Tests

- TC1: Multi-device token validation

Design

- Store session metadata

Plan

- Build session table

Epic 2: Remediation & Guidance

1. As a Farmer, I want to receive clear step-by-step remediation instructions after a disease is detected, so that I know exactly what actions to take to protect my crop.

AC: System shows ordered remediation steps only when disease is detected.

Plan:

Fetch disease-specific remediation steps from a structured knowledge base once diagnosis is confirmed.

Design:

- Logic: Map disease_name → remediation_steps[].
- UI: Vertical stepper component showing one action per step.
- Accessibility: Simple wording with icons (spray, water, warning).

Test:

- Detect disease → Verify remediation steps appear

- Verify steps are ordered logically
- Verify no steps shown if plant is “Healthy”

2. As a Farmer, I want to see both chemical and organic treatment options, so that I can choose based on cost, availability, or preference.

AC: System allows switching between chemical and organic treatments.

Plan:

Store remediation strategies under two categories: Chemical and Organic.

Design:

- Data Model:


```
{
  "Chemical": [...],

  "Organic": [...]
}
```
- UI: Tab-based switch between “Chemical” and “Organic”.

Test:

- Select “Chemical” → Only chemical steps shown
- Switch to “Organic” → Organic steps load correctly

3. As a Farmer, I want the system to recommend dosage and application frequency, so that I don’t overuse or underuse treatments.

Plan:

Include dosage rules based on crop type and disease severity.

Design:

- Logic: Severity-based dosage multiplier
- UI: Dosage card highlighted in bold
- Safety: Display “Do not exceed” warning

Test:

- Mild severity → Lower dosage shown
- Severe severity → Higher dosage shown

4. As a Farmer, I want to see safety warnings and precautions, so that I can protect myself, soil, and nearby crops.

Plan:

Attach safety metadata to each remediation step.

Design:

- UI: Warning banner with ⚠ icon
- Content: PPE requirements, water contamination warnings

Test:

- Chemical treatment → Safety warning visible
- Organic treatment → Reduced but present warnings

5. As a Farmer, I want the remediation steps to be shown in my native language, so that I can clearly understand the instructions.

Plan:

Translate remediation text using a predefined language dictionary.

Design:

- Logic: language_code → translated_steps
- Fallback: Default to English if translation missing

Test:

- Select Kannada/Tamil → Steps translated
- Remove translation → English fallback works

6. As a Low-Literacy User, I want voice-based instructions, so that I can follow guidance without reading long text.

Plan:

Use text-to-speech to narrate remediation steps.

Design:

- TTS Engine: Native mobile TTS
- UI: Play/Pause audio button per step

Test:

- Tap “Play” → Audio narration starts
- Switch language → Voice changes accordingly

7. As a Farmer, I want the system to recommend preventive measures, so that the disease does not spread or reoccur.

AC: Disease-specific prevention tips are provided.

Plan:

Attach prevention guidelines linked to each disease.

Design:

- UI: “Prevention Tips” collapsible section
- Content: Crop rotation, spacing, irrigation advice

Test:

- After treatment → Prevention section visible
- Verify prevention tips match disease

8. As a Farmer, I want remediation guidance to change based on disease severity, so that early-stage issues are handled gently and critical ones urgently.

Plan:

Use severity output from detection module to branch guidance logic.

Design:

- Logic:
 - Low → Organic-first advice
 - Medium → Combined approach
 - High → Immediate chemical intervention
- UI: Severity badge shown at top

Test:

- Low severity → Organic shown first
- High severity → Chemical prioritized

9. As a Farmer, I want the system to tell me when no treatment is needed, so that I don't waste money unnecessarily.

AC: For healthy or low-risk detections, the system must clearly indicate that no treatment is required.

Plan:

If disease is "Healthy" or "Low-risk", skip remediation.

Design:

- Logic: If severity < threshold → No remediation
- UI: Green confirmation message

Test:

- Healthy leaf → "No treatment required" shown
- No chemical suggestions appear

10. As a Farmer, I want offline access to remediation guidance, so that I can still use the app in low-network areas.

Plan:

Cache remediation steps locally after first fetch.

Design:

- Storage: Local JSON cache
- Logic: Load cached data when offline

Test:

- Online → Data fetched and cached
- Offline → Cached guidance loads

11. As a Farmer, I want to see estimated cost ranges for treatments, so that I can decide what is affordable.

Plan:

Store approximate cost ranges for remediation items.

Design:

- UI: ₹ Low / ₹₹ Medium / ₹₹₹ High indicator
- Logic: Cost mapped per treatment type

Test:

- Chemical treatment → Higher cost shown
- Organic → Lower cost indicator

12. As a Farmer, I want remediation steps to be shown as icons and visuals, so that they are easier to follow.

Plan:

Associate each step with a simple visual icon.

Design:

- UI: Icon + short text per step
- Icons: Spray, Water, Wait, Warning

Test:

- Each step displays correct icon
- Icons remain consistent across languages

13. As a User, I want the system to suggest consulting an expert when the disease is severe or unclear, so that I don't rely blindly on automation.

Plan:

Add escalation logic for high-risk cases.

Design:

- Logic: If severity = High OR confidence < threshold
- UI: "Consult Expert" CTA button

Test:

- Severe disease → Expert suggestion shown
- Mild disease → No escalation

14. As a Farmer, I want to access community-recommended remedies, so that I can learn from nearby farmers' experiences.

AC: Community-suggested remedies must be displayed separately as non-verified advisory information.

Plan:

Integrate community suggestions as optional, non-primary guidance.

Design:

- UI: "Community Tips" section (read-only)
- Moderation: Marked as "Not AI-verified"

Test:

- Community tips visible

- Clearly labeled as advisory only

Epic 3: AI model for disease detection

1. As a Farmer, I want to upload an existing photo from my phone's gallery, so that I can analyze a picture I took earlier in the field.

AC: The system must accept standard image formats (JPG, PNG, HEIC).

Plan: Implement a permission-aware file picker.

Design:

- **UI:** A "Gallery" icon button on the dashboard.
- **Logic:** Use image picker library to access local storage.

Test: Select a valid JPG (Pass); Select a PDF (Fail/Greyed out); Deny storage permission (Check error handling).

2. As a Field Worker, I want to capture an image directly using the app's camera interface, so that I can quickly diagnose a plant without switching apps.

AC: After capturing, the user must see a preview with "Retake" and "Use Photo" options.

Plan: Integrate a camera view inside the app, rather than calling the external camera app.

Design:

- **Architecture:** Use a CameraController to manage the stream
- **UI Flow:** Live Preview -> Capture Button -> Frozen Frame Overlay-> Confirmation Dialog.

Test: Verify camera initializes; Capture photo; Click "Retake" (camera restarts); Click "Use" (image path returned).

3. As a Farmer, I want to record a short 5-second video of the plant, so that the AI can analyze multiple angles and frames to improve accuracy.

Plan: Record a short video, extract keyframes, average the prediction results

Design:

- **Processing Pipeline:** Record video -> Use ffmpeg or native code to extract 3 frames
- **Logic:** use a "Majority Voting" algorithm to determine the final result.

Test: Record for 5 seconds; Verify 3 distinct images are generated; Verify final result matches the most common disease detected.

4. As a User, I want to select the specific crop type (e.g., Tomato, Rice, Cotton) from a menu, so that the system uses the correct disease model for analysis.

Plan: Implement a pre-analysis menu, where the farmer can select the crop.

Design:

- **Data Structure:** A JSON map
- **State Management:** unload current interpreter and initialize the new one

Test: Select 'Corn'; Verify 'corn_model' is loaded in logs; Select 'Rice'; Verify model switches.

5. As a User, I want the app to automatically detect if my photo is too blurry and prompt me to retake it, so that I don't get an incorrect result due to bad image quality.

Plan: Preprocess the image and check the quality of the image

Design:

- **Algorithm:** Calculate the **Variance of the Laplacian**
- **Thresholding:** If variance < 100, flag as blurry and show a warning.

Test: Take a photo while shaking the phone (Blur detected); Take a stable photo (Pass).

6. As a Farmer, I want to see a box drawn around the specific infected area on the leaf, so that I know exactly which spot the AI is analyzing.

AC: If multiple spots are detected, multiple boxes should be drawn.

Plan: get coordinates and display them on the image

Design:

- **Rendering:** Use a CustomPainter to draw rectangles

- **Coordinate Mapping:** Convert model output to screen pixel dimensions

Test: Use an image with 3 distinct spots; Verify 3 separate boxes appear; Verify boxes align with the spots.

7. As a Farmer, I want to see a percentage confidence score (e.g., "92% confident") next to the result, so that I know how much to trust the AI's judgment.

Plan: Extract and display the probability array from the models output layer

Design:

- **UI:** Color-coded badge component: Green (>80%), Yellow (50-80%), Red (<50%).

Test: Mock model output 0.95 (Check Green "95%"); Mock 0.40 (Check Red "40%").

8. As an Agronomy Student, I want to view a "heatmap" overlay on my image, so that I can understand which visual patterns the AI used to identify the disease (Explainable AI).

Plan: Implement a heatmap and display it alongside the image

Design:

- **Logic:** Extract gradients from the last convolutional layer
- **UI:** Generate a heatmap

Test: Upload image of leaf with tip burn; Verify red "hot" zone covers the tip burn area.

9. As a Farmer, I want the system to explicitly tell me if the plant is "Healthy", so that I don't waste money buying medicine for a problem that doesn't exist.

Plan: Include a "healthy" class while training

Design:

- **Logic:** if (predicted_class == "Healthy") trigger the "Success" workflow.
- **UI:** Display a large Green Shield icon

Test: Upload a pristine leaf; Verify "Healthy" badge appears; Verify no pesticide recommendations are shown.

10. As a User, I want the diagnosis to indicate the severity level (e.g., Early Stage vs. Critical), so that I can decide if I need immediate chemical intervention or mild organic care.

Design:

- **Algorithm:** $\text{Severity} = (\text{Sum of Bounding Box Areas}) / (\text{Total Leaf Area})$.
- **Logic:** $<10\% = \text{Low}$; $10-40\% = \text{Medium}$; $>40\% = \text{High}$.

Test: Upload image with small spot (Check "Low"); Upload fully brown leaf (Check "Critical").

11. As a Farmer, I want the system to identify if a single plant has two different infections simultaneously, so that I can treat both problems at once.

AC: The UI must display "Primary Diagnosis" and "Secondary Diagnosis" clearly

Plan: Use a certain probability threshold for all classes, not just one

Design:

- **Filtering:** Loop through all class probabilities; keep all where score $>$ threshold
- **UI:** Sort results by confidence

Test: Upload image with Late Blight and Mites; Verify both appear in the result list.

12. As a Farmer, I want to manually crop and zoom into the specific infected part of the leaf before analysis, so that I can force the AI to focus on the disease spot rather than the background soil or sky.

Plan: Integrate a minimal image editing tool

Design:

- **Library:** Integrate image_cropper.
- **Data Flow:** Original Image $\rightarrow$ User Crop $\rightarrow$ AI Model.

Test: Take photo of whole plant; Crop to single leaf; Verify result is specific to that leaf.

13. As a User, I want the system to return an "Unknown/Unclear" result instead of a random guess, so that I am not misled when the AI encounters a new or rare disease.

AC: The system must advise the user to consult a human expert or retake the photo.

Plan: Use a rejection threshold

Design:

- **Logic:** If (Max_Confidence_Score < 0.45) then return Result.Unknown.

Test: Upload a photo of a shoe; Verify system says "Unknown" (or low confidence) rather than predicting disease.

14. As a Farmer, I want to see a side-by-side comparison of my photo against a "reference image" of the diagnosed disease, so that I can visually confirm if they look similar.

Plan: Maintain a local database of reference image

Design:

- **UI:** "Split View" widget
- **Interaction:** to allow detailed texture comparison.

Test: Diagnose "Early Blight"; Verify the reference image shown is correctly labeled "Early Blight".

EPIC 4: Native Language Support

USER STORY 1 — Auto-Detect User Language

As a farmer, I want the app to auto-detect my language from voice input so I don't have to configure settings.

Acceptance Criteria

- System identifies the language with 90% accuracy.
- Suggests correct language to user for confirmation.

Testing

- Test voice samples of different languages.
- Test noisy environments.

Design

- Use Google Language Identification API or Whisper LID.
- Show confirmation popup.

Plan

- Build LID module → integrate popup → test with all languages.

USER STORY 2 — Text Input in Native Language

As a farmer, I want to type my questions in my own language so communication is easy.

✓ Acceptance Criteria

- Input field supports Unicode typing.
- App correctly processes native text.

Testing

- Type using multiple languages.
- Check compatibility with English + native mixed input.

Design

- Unicode-friendly font stack.
- Native keyboard support.

Plan

- Enable UTF-8 → test multi-language input → integrate with NLP pipeline.

USER STORY 3 - Voice Input in Native Language

As a farmer, I want to speak in my native language so I don't need to type.

✓ Acceptance Criteria

- Voice recognition supports multiple Indian languages.

- Converts speech to text reliably.
- Handles accents.

Testing

- Test in varied noise levels.
- Test multiple accents & speed.

Design

- Use ASR (Google Speech/Whisper).
- Mic button UI.

Plan

- Integrate ASR → test with real farmers → optimize accuracy.

USER STORY 4 — Voice Output (Text-to-Speech)

As a farmer, I want the solution to read the disease diagnosis in my native language so I don't need to read text.

Acceptance Criteria

- TTS available for all languages.
- Natural, clear voice output.

Testing

- Test speed, pitch, clarity.
- Test offline conditions (if supported).

Design

- Use TTS API models (Google TTS, Azure).
- Play/pause controls.

Plan

- Add TTS module → test → integrate into diagnosis screen.

USER STORY 5 — Simplified Local Vocabulary Mode

As a farmer, I want the app to use simple, farmer-friendly words in my language instead of technical terms.

✓ Acceptance Criteria

- Replace complex agricultural jargon.
- All instructions become easy to follow.

Testing

- Validate vocabulary with agricultural domain experts.

Design

- Maintain glossary mapping (Technical → Simple).
- Option for “Simple Mode”.

Plan

- Build glossary → integrate into translation → test with users.

USER STORY 6 — Multi-Language Mixed Input

As a farmer, I want to enter a mix of English and native language so the app understands hybrid sentences.

✓ Acceptance Criteria

- NLP pipeline processes code-mixed text (e.g., Hinglish).
- No errors when switching languages.

Testing

- Test “Hinglish”, “Tanglish”, “Kanglish”.

Design

- Language detection per-word.
- Combined vocab model.

Plan

- Train code-mixed model → test → optimize

USER STORY 7 — Local Script Font Rendering

As a user, I want all languages to render clearly so text remains readable.

Acceptance Criteria

- No text clipping.
- No box characters (□).
- Proper rendering on low-end devices.

Testing

- Test on different screen sizes.
- Test different fonts.

Design

- Use Noto Sans fonts for Indian languages.

Plan

- Integrate fonts → optimize UI.

USER STORY 8 — Native Language Tutorials

As a farmer, I want video/audio tutorials in my language so I understand how to use the app.

✓ Acceptance Criteria

- Tutorials available in all languages.

- Auto-play voice instructions.

Testing

- Playback quality.
- Language selection accuracy.

Design

- Media player UI.
- Auto-play narration.

Plan

- Record scripts → produce videos → integrate into Help section.

USER STORY 9 — Multi-Language Notifications

As a farmer, I want disease alerts to arrive in my native language so I understand risks instantly.

Acceptance Criteria

- Notifications appear fully translated.
- No English fallback (except technical errors).

Testing

- Test push alerts in each language.

Design

- Notification templates using i18n keys.

Plan

- Create templates → integrate → test.

USER STORY 10 — Native Language OCR

As a farmer, I want the app to read printed local-language labels or leaflets so I can upload info easily.

✓ Acceptance Criteria

- OCR recognizes Indian scripts.
- Extracted text is accurate.

Testing

- Test with low-light images.
- Test with multiple fonts.

Design

- Use Tesseract or Azure OCR.
- Input image pre-processing.

Plan

- Add OCR → test → refine

USER STORY 11 — Native Language Error Message

As a farmer, I want all error messages in my language so I understand what went wrong.

✓ Acceptance Criteria

- Clear, actionable errors.
- No untranslated words.

Testing

- Trigger errors intentionally.
- Verify translations.

Design

- i18n error keys.

- Simple phrasing.

Plan

- Build error dictionary → integrate → QA.

USER STORY 12 — Offline Language Pack Support

As a farmer in remote areas, I want local language support even without internet so I can still use the app.

Acceptance Criteria

- Language pack stored offline.
- ASR/TTS fallback to on-device models.

Testing

- Turn off internet and test all flows.

Design

- Downloadable language pack.
- Light-weight offline TTS/STT.

Plan

- Build offline mode → integrate LLM-lite → test on low-end devices.

USER STORY 13 — Language Preference Switching During Interaction

User Story

As a farmer, I want to switch my language at any time during usage so that I can continue comfortably if my preference changes.

✓ Acceptance Criteria

- User can change language without restarting the app.
- Current screen content updates immediately to the new language.
- Ongoing chatbot or voice interaction continues smoothly.
- Previously saved data remains unaffected.

Testing

- Switch language during:
 - Chatbot conversation
 - Voice playback
 - Image diagnosis result display
- Verify no crashes or data loss.
- Validate persistence after app restart.

Design

- Global language state manager.
- Dynamic UI re-rendering using i18n keys.
- Language switch icon always accessible.
- Session-aware chatbot language update.

Plan

1. Implement global language state handling.
2. Enable real-time re-rendering.
3. Integrate session-level language updates.
4. Test mid-session switching scenarios.
5. Optimize UX performance.

USER STORY 14 — Native Language Feedback & Correction

As a farmer, I want to give feedback or correct the AI's language response in my native language so that the system improves over time.

✓ Acceptance Criteria

- User can submit feedback in text or voice in their native language.
- Feedback can indicate:
 - Incorrect translation
 - Unclear wording
 - Wrong pronunciation
- System stores feedback securely.
- No feedback action disrupts current user flow.

Testing

- Submit feedback in multiple languages.
- Test voice-based feedback.
- Verify feedback is stored correctly.
- Ensure feedback submission works offline (queued for sync).

Design

- “Report / Improve Language” option on responses.
- Simple rating + comment interface.
- Feedback tagging (translation, clarity, pronunciation).
- Secure backend storage.

Plan

1. Design feedback UI for text & voice.
2. Integrate native language input handling.
3. Store feedback with language metadata.
4. Sync feedback to cloud when online.
5. Use feedback to refine language models.

- EPIC 5:
Offline Model and Low Network Optimization

US-01: Local Image Classification Without Internet

User Story:

As a farmer, I want to detect crop disease from images without internet, so that I can take quick action in the field.

Acceptance Criteria:

Prediction in under 3 seconds, supports JPG/PNG images, shows confidence score.

Testing:

Test accuracy on different devices and image quality.

Design:

Camera/gallery input with result display.

Plan:

Use TFLite for on-device inference.

US-02: Offline Recommendation Sync

User Story:

As a farmer, I want recommendations saved offline and synced later, so that no treatment data is lost.

Acceptance Criteria:

Data stored locally and auto-synced without duplicates.

Testing:

Test sync after internet reconnect.

Design:

Sync indicator with pending count.

Plan:

SQLite storage with sync manager.

US-03: Automatic Data Sync

User Story:

As a farmer, I want data to be **automatically synced**, so that important information reaches the server as soon as internet is available.

Acceptance Criteria:

Data syncs automatically in the background and shows sync status (syncing / success / failed).

Testing:

Test automatic sync on different network speeds and network availability changes.

Design:

Background sync indicator (icon/status message).

Plan:

Automatic sync trigger based on network availability.

US-04: Network Status Indicator

User Story:

As a farmer, I want to see network status, so that I know when the app is online or offline.

Acceptance Criteria:

Shows online, offline, weak connection.

Testing:

Test during network changes.

Design:

Status bar indicator.

Plan:

Network monitoring module.

US-05: Offline Weather Data

User Story:

As a farmer, I want weather data available offline, so that I can plan farming activities anytime.

Acceptance Criteria:

Stores recent weather data locally.

Testing:

Test data availability without internet.

Design:

Offline weather widget.

Plan:

Weather API with local caching.

US-06: Offline Disease Database

User Story:

As a farmer, I want disease information offline, so that I can access guidance in remote areas.

Acceptance Criteria:

Disease data downloadable and accessible offline.

Testing:

Test download and offline access.

Design:

Download screen with list view.

Plan:

Compressed disease database.

US-07: Adaptive Image Compression

User Story:

As a farmer, I want images compressed automatically, so that uploads work on slow networks.

Acceptance Criteria:

Image size reduced without accuracy loss.

Testing:

Test upload speed and accuracy.

Design:

Auto compression in background.

Plan:

Adaptive compression logic.

US-08: Batch Upload with Resume

User Story:

As a farmer, I want batch uploads with resume, so that network issues don't stop uploads.

Acceptance Criteria:

Uploads resume after interruption.

Testing:

Simulate network drop during upload.

Design:

Upload queue with progress.

Plan:

Chunked upload implementation.

US-09: Offline Feature Indicator

User Story:

As a farmer, I want to know which features work offline, so that I can use the app effectively.

Acceptance Criteria:

Offline features clearly marked.

Testing:

Test feature behavior offline.

Design:

Badges and tooltips.

Plan:

Feature classification.

US-10: Secure Local Data Storage

User Story:

As a farmer, I want my data encrypted, so that it remains secure.

Acceptance Criteria:

Local data encrypted.

Testing:

Test data access and security.

Design:

Transparent encryption.

Plan:

Use encrypted local database.

US-11: Location-Based Prefetching

User Story:

As a farmer, I want nearby data prefetched, so that the app works faster offline.

Acceptance Criteria:

Prefetch only on WiFi.

Testing:

Test location-based data fetch.

Design:

Prefetch settings screen.

Plan:

GPS-based scheduler.

US-12: Offline Search and Filter

User Story:

As a farmer, I want offline search, so that I can quickly find disease info.

Acceptance Criteria:

Search works without internet.

Testing:

Test search speed offline.

Design:

Search bar with filters.

Plan:

Local database search.

US-13: Offline Audit Trail

User Story:

As a farmer, I want record history offline, so that I can track changes.

Acceptance Criteria:

Changes logged locally.

Testing:

Verify history accuracy.

Design:

Timeline view.

Plan:

Audit logging module.

US-14: Low Bandwidth Support

User Story:

As a farmer, I want the app usable on weak networks, so that core features always work.

Acceptance Criteria:

Core features functional on low bandwidth.

Testing:

Test on slow networks.

Design:

Lightweight UI mode.

Plan:

Bandwidth-based feature control.